

Függvények Pythonban: def és return

Jegyzet és feladatlap

Hogyan dolgozz ezzel a jegyzettel?

Ez a jegyzet úgy épül fel, hogy egyedül is végig tudj menni rajta. Három dolog segít ebben:

- **Kis lépések.** Minden újdonságot előbb egy kész kódon próbálsz ki, és csak utána írsz sajátot.
- **Ellenőrzés.** A feladatok megadják, mit kell kapnod. Ha a programod ugyanazt írja ki, jó úton jársz, és mehetsz tovább.
- **Hibakereső.** Ha hibaüzenetet kapsz, lapozz a jegyzet végére. A Hibakereső táblázat megmondja, mit jelent az üzenet, és mit tegyél.

A jelölések a szokásosak: a **Próbáld ki** kódjait gépeld be és futtasd, a **Tippelj** részeknél előbb írd le a tippedet, a **Feladat** pedig a te munkád. A nehézséget csillagok jelzik: ★ = alap, ★★ = közepes, ★★★ = nehezebb.

Ha elakadsz, ebben a sorrendben próbálkozz:

1. Olvasd el újra a feladat előtti mintakódot. A megoldás szinte mindig annak az átalakítása.
2. Nézd meg a Hibakeresőt a jegyzet végén.
3. Kérdezd meg a padtársadat. A C++-os társaid ugyanezeket a feladatokat oldják meg, ugyanezzel a számozással.
4. Ha ez sem segített, szólj a tanárodnak.

1. Az első saját függvény

A függvény egy névvel ellátott programrészlet. Egyszer megírod, és ahányszor kell, a nevével meghívod.

Próbáld ki - ismétlődő kód

```
print("-----")
print("Informatika szakkor")
print("-----")
print("Mai tema: fuggvenyek")
print("-----")
```

Futtasd, aztán cseréld le mindhárom vonalban a kötőjeleket egyenlőségjelekre. Hány sort kellett átírnod?

Próbáld ki - ugyanez függvénnyel

```
def vonal():
    print("-----")

vonal()
print("Informatika szakkor")
vonal()
print("Mai tema: fuggvenyek")
vonal()
```

Ugyanazt írja ki, mint az előző program. Most is cseréld le a kötőjeleket egyenlőségjelekre. Hány sort kellett átírnod?

Nézzük meg a függvény részeit:

- **def** – ezzel a szóval kezdődik minden függvény megírása (definíciója).
- **von**al – a függvény neve. Te választod, mint egy változót.
- **()** – ide kerülnek majd a paraméterek. Most üres.
- **:** – a sor végén kettőspont áll, ahogy a **for** és az **if** után.
- A behúzott sorok alkotják a függvény törzsét: ezek az utasítások futnak le.
- **von**al**()** – a **hívás**. Ennél a sornál hajtja végre a program a függvény törzsét, aztán a következő sorral folytatja.

A függvények alatti, be nem húzott részt **főprogramnak** hívjuk. A program futása itt kezdődik.

Tippelj – milyen sorrendben fut?

Írd le a füzetedbe, milyen betűk jelennek meg, és milyen sorrendben. Csak utána futtasd!

```
def elso():  
    print("A")  
  
def masodik():  
    print("B")  
  
print("C")  
masodik()  
elso()  
masodik()
```

A függvények abban a sorrendben futnak le, ahogy meghívod őket – nem abban, ahogy megírtad.

Kísérlet – függvény hívás nélkül

Töröld ki az előző programból az **elso()** sort. Megjelenik még az **A** betű? Fogalmazd meg egy mondatban a szabályt.

Most írd vissza a sort, de zárójelek nélkül: **elso**. Kapsz hibaüzenetet? Megjelenik az **A**?

Figyelem! A hívásnál mindig ki kell tenni a zárójelet, akkor is, ha üres. Zárójel nélkül a Python nem jelez hibát, de a függvény nem fut le.

Kísérlet – hol legyen a függvény?

Vágd ki az **elso** függvényt (mindkét sort), és illeszd be a program legvégére. Mi történik? Olvasd el a hibaüzenet utolsó sorát, és keresd meg a Hibakeresőben.

Figyelem! A saját függvényeidet mindig a program elejére írd, a főprogram kerüljön alájuk. A Python felülről lefelé halad: amit még nem látott, azt nem ismeri.

Feladatok

1.1. feladat ★☆☆ - Üdvözlés

Fejezd be ezt a programot. Az `udvozles` függvény két sort írjon ki (`Szia!` és `Jo munkat!`), a főprogram pedig hívja meg háromszor.

```
def udvozles():  
    print("Szia!")  
    # ide írd a második kiírást  
  
# itt hívd meg háromszor
```

Ellenőrzés: hat sornak kell megjelenennie, felváltva `Szia!` és `Jo munkat!`.

1.2. feladat ★☆☆ - Csillagsor

Írj `csillagsor` nevű függvényt, amely tíz csillagot ír ki egy sorba. Hívd meg kétszer, és a két hívás között írd ki a nevedet.

Ellenőrzés:

```
*****  
Anna  
*****
```

1.3. feladat ★☆☆ - Három függvény

Írj három függvényt: `eleje`, `kozepe`, `vege`. Mindegyik egy sort ír ki: `Egyszer volt, - hol nem volt, - volt egyszer egy program`. A főprogram hívja meg őket úgy, hogy a mese jó sorrendben jelenjen meg.

Ellenőrzés: három sor, a fenti sorrendben. Ezután cseréld fel a hívásokat a főprogramban. A függvényekhez hozzá kellett nyúlnod?

1.4. feladat ★☆☆ - Függvény a függvényben

Írj `fejlec` nevű függvényt, amely meghívja a `csillagsor` függvényt, kiírja az `INFORMATIKA` szót, majd újra meghívja a `csillagsor`-t. A főprogramban most csak egyetlen sor legyen: a `fejlec()` hívás.

Ellenőrzés: ugyanaz a három sor, mint az 1.2. feladatban, csak a név helyén `INFORMATIKA` áll.

2. Paraméterek: adat a függvénynek

A paraméterrel adatot adsz át a függvénynek. Így ugyanaz a függvény minden hívásnál mást csinálhat.

Próbáld ki

```
def vonal(hossz):  
    print("-" * hossz)  
  
vonat(5)  
vonat(20)  
  
n = 12  
vonat(n)  
vonat(n // 2)
```

A zárójelben álló `hossz` a **paraméter**: egy változó, amely a híváskor kapja meg az értékét. A `vonat(5)` hívásnál a `hossz` értéke 5, a `vonat(20)` hívásnál 20.

Híváskor a zárójelbe szám, változó vagy számítás is kerülhet. Milyen hosszú vonalat rajzolt az utolsó hívás?

Próbáld ki - több paraméter

Cseréld le a `vonalt` függvényt erre a változatra:

```
def vonal(hossz, jel):  
    print(jel * hossz)
```

A főprogramban hívd így: `vonalt(10, "*")` és `vonalt(3, "#")`

A paramétereket vessző választja el.

Tippelj - számít a sorrend?

```
def kivon(a, b):  
    print(a - b)  
  
kivon(10, 3)  
kivon(3, 10)  
  
a = 1  
b = 8  
kivon(b, a)
```

A harmadik hívás a trükkös. Az értékek a **helyük** szerint kerülnek a paraméterekbe, nem a nevük szerint.

Próbáld ki - sztring paraméter

```
def koszon(nev):  
    print(f"Szia, {nev}!")  
  
koszon("Anna")  
  
valaki = input()  
koszon(valaki)
```

Kísérlet - hibás hívások

Írd be az alábbi sorokat **egyesével** a kétparaméteres `vonalt` függvényt használó programod végére. Olvasd el a hibaüzenet utolsó sorát, keresd meg a Hibakeresőben, aztán töröld a hibás sort.

```
vonalt()  
vonalt(5, "*", 3)  
vonalt("ot", "*")
```

Végül próbáld ki ezt is: `vonalt(3, 7)`. Hibát vártál? Mit írt ki, és miért?

Figyelem! A Python nem ellenőrzi, milyen típusú adatot adsz át. Ha számot adsz szöveg helyett, a függvény nem biztos, hogy hibát jelez – lehet, hogy csak mást csinál, mint amit szeretnél.

Feladatok

2.1. feladat ★☆☆ - Köszönés névvel

Gépeld be a `koszon` függvényt, és hívd meg három különböző névvel.

Ellenőrzés: három sor, például `Szia, Bence!`

2.2. feladat ★☆☆ - Számsor

Írj `szamsor(n)` függvényt, amely kiírja a számokat 1-től `n`-ig egy sorba, szóközzel elválasztva.

Ellenőrzés: a `szamsor(5)` hívás kimenete `1 2 3 4 5`, a `szamsor(1)` hívásé `1`.

2.3. feladat ★☆☆ - Szorzótábla

Írj `szorzotabla(n)` függvényt, amely kiírja az `n` szorzótábláját 1-től 10-ig.

Ellenőrzés: a `szorzotabla(7)` hívás első sora $1 \times 7 = 7$, az utolsó $10 \times 7 = 70$.

2.4. feladat ★☆☆ - Téglalap

Írj `teglalap(szelesseg, magassag)` függvényt, amely csillagokból téglalapot rajzol. **Tipp:** a függvényed meghívhatja a kétparaméteres `vonal` függvényt.

Ellenőrzés: a `teglalap(5, 3)` hívás kimenete:

```
*****
*****
*****
```

2.5. feladat ★☆☆ - Háromszög

Írj `haromszog(n)` függvényt: az első sorban egy, a másodikban kettő, az utolsóban `n` csillag áll.

Ellenőrzés: a `haromszog(4)` hívás kimenete:

```
*
**
***
****
```

2.6. feladat ★☆☆ - Bemutatás

Írj `bemutat(nev, kor)` függvényt.

Ellenőrzés: a `bemutat("Anna", 17)` hívás kimenete `Anna 17 éves, jövőre 18 lesz.`

3. Visszatérési érték: a függvény válaszol

A `return` utasítással a függvény eredményt ad vissza oda, ahonnan meghívták.

Próbáld ki

```
def negyzet(x):
    return x * x

print(negyzet(5))

eredmeny = negyzet(7)
print(eredmeny)
```

Mi változott az eddigiekhez képest?

- A függvényben nincs `print`, helyette `return` áll, utána pedig az eredmény.
- A hívás helyére az eredmény kerül. Képzeld úgy, hogy a programban a `negyzet(5)` helyén a 25 áll.

Tippelj

Az eredménnyel bármit megtehetsz, amit egy számmal. Mit ír ki ez a részlet?

```
print(negyzet(3) + negyzet(4))
print(negyzet(negyzet(2)))

a = 6
if negyzet(a) > 30:
    print("nagy")
```

Kísérlet - kiírni vagy visszaadni?

Ez a fejezet legfontosabb kísérlete. Legyen a programodban mindkét függvény:

```
def kiir_negyzet(x):  
    print(x * x)  
  
def negyzet(x):  
    return x * x
```

Írd be ezeket a sorokat **egyesével** a főprogramba, és figyeld meg, mi történik:

```
kiir_negyzet(5)  
negyzet(5)  
a = negyzet(5) + 1  
b = kiir_negyzet(5) + 1  
print(kiir_negyzet(5))
```

Melyik sor írt ki valamit? Melyik után nem történt semmi látható? Melyik hibás? Mit jelent a **None** az utolsó sor kimenetében?

Figyelem! A kiírás és a visszaadás nem ugyanaz! Amit a függvény kiír, az megjelenik a képernyőn, de a program nem tud vele tovább számolni. Amit visszaad, azzal a hívó azt tesz, amit akar. Ha egy függvényben nincs **return**, az eredménye **None**, vagyis „semmi”.

Tippelj - mi történik a return után?

```
def abszolot(x):  
    if x < 0:  
        return -x  
    return x
```

Kövesd végig papíron az **abszolot(-4)** és az **abszolot(9)** hívást. Melyik **return** fut le az egyik, és melyik a másik esetben? Utána írasd ki mindkettőt.

A **return** azonnal befejezi a függvényt. Ami utána áll, az már nem fut le.

Kísérlet - ha elmarad a return

Írd át a **negyzet** függvény **return x * x** sorát erre: **x * x** (a **return** szó nélkül). Írasd ki a **negyzet(5)** értékét. Mit kapsz?

Feladatok

3.1. feladat ★☆☆ - Duplázó

Írj **duplaz(x)** függvényt, amely visszaadja a szám kétszeresét. A főprogramban írasd ki az eredményt.

Ellenőrzés: **duplaz(4)** → 8, **duplaz(-3)** → -6, **duplaz(0)** → 0.

3.2. feladat ★☆☆ - Összeg

Írj **osszeg(a, b)** függvényt.

Ellenőrzés: **osszeg(3, 4)** → 7, **osszeg(-5, 5)** → 0.

3.3. feladat ★☆☆ - Téglalap adatai

Írj két függvényt: a **terulet(a, b)** a téglalap területét, a **kerulet(a, b)** a kerületét adja vissza.

Ellenőrzés: **terulet(3, 5)** → 15, **kerulet(3, 5)** → 16.

3.4. feladat ★☆☆ - Nagyobb

Írj `nagyobb(a, b)` függvényt, amely a két szám közül a nagyobbbat adja vissza. A beépített `max` most nem használható.

Ellenőrzés: `nagyobb(3, 7) → 7`, `nagyobb(7, 3) → 7`, `nagyobb(5, 5) → 5`.

3.5. feladat ★☆☆ - Legnagyobb a három közül

Írj `legnagyobb(a, b, c)` függvényt. Ne írd új `if`-eket: használd fel a `nagyobb` függvényt!

Ellenőrzés: `legnagyobb(2, 9, 4) → 9`, `legnagyobb(9, 2, 4) → 9`, `legnagyobb(2, 4, 9) → 9`.

3.6. feladat ★☆☆ - Átlag

Írj `atlag(a, b)` függvényt, amely két szám átlagát adja vissza.

Ellenőrzés: `atlag(3, 4) → 3.5`, `atlag(10, 20) → 15.0`. Ha az első esetben 3-at kapsz, nézd meg, melyik osztásjelet használtad.

3.7. feladat ★☆☆ - Hőmérséklet

Írj `fahrenheit(celsius)` függvényt. A képlet: $F = C \cdot 9 / 5 + 32$.

Ellenőrzés: `fahrenheit(0) → 32.0`, `fahrenheit(100) → 212.0`, `fahrenheit(37) → 98.6` (vagy egy ehhez nagyon közeli szám).

3.8. feladat ★☆☆ - Faktoriális

Írj `faktorialis(n)` függvényt. Az `n` faktoriálisa a számok szorzata 1-től `n`-ig.

Ellenőrzés: `faktorialis(5) → 120`, `faktorialis(1) → 1`, `faktorialis(10) → 3628800`.

4. Igaz vagy hamis? Logikai függvények

Egy logikai függvény egy kérdésre válaszol igennel vagy nemmel: `True` vagy `False` értéket ad vissza. Ettől a feltételeid olvashatóbbak lesznek.

Próbáld ki

```
def paros(x):  
    return x % 2 == 0  
  
n = int(input())  
  
if paros(n):  
    print("paros")  
else:  
    print("paratlan")
```

Az `x % 2 == 0` kifejezés értéke maga `True` vagy `False`. A függvény ezt adja vissza.

Olvasd fel hangosan az `if` sorát: „ha páros `n`”. Ezért érdemes a logikai függvénynek kérdésre válaszoló nevet adni.

Kísérlet - mit ír ki?

```
print(paros(4), paros(7))
```

Próbáld ki - keresés függvényben

Ez a függvény azt dönti el, van-e a számnak 1-en és önmagán kívül más osztója:

```
def osszetett(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return True  
    return False
```

Kövesd végig papíron $n = 9$ és $n = 7$ esetén. Hányadik körben áll meg a függvény az egyik, és hányadikban a másik esetben? Utána írasd ki: `osszetett(9)`, `osszetett(7)`, `osszetett(15)`.

Figyeld meg a szerkezetet: amint találtunk egy osztót, a `return True` azonnal kilép. A `return False` a ciklus **után** áll (nézd meg a behúzását!) – oda csak akkor jutunk el, ha egyetlen osztót sem találtunk. Ugyanez volt a szabály a listás keresésnél is.

Feladatok

4.1. feladat ★☆☆ - Pozitív-e?

Írj `pozitiv(x)` függvényt.

Ellenőrzés: `pozitiv(5)` → True, `pozitiv(-2)` → False, `pozitiv(0)` → False.

4.2. feladat ★☆☆ - Osztható-e?

Írj `oszthato(a, b)` függvényt, amely megmondja, osztható-e a a b számmal.

Ellenőrzés: `oszthato(12, 4)` → True, `oszthato(12, 5)` → False.

4.3. feladat ★☆☆ - Nagykorú

Írj `nagykoru(kor)` függvényt. A főprogramban olvass be egy életkort, és a függvény segítségével írd ki, hogy az illető nagykorú-e.

Ellenőrzés: `nagykoru(18)` → True, `nagykoru(17)` → False.

4.4. feladat ★☆☆ - Határok között

Írj `kozte(x, also, felso)` függvényt: igaz, ha x az `also` és a `felso` közé esik. A határok is beleszámítanak.

Ellenőrzés: `kozte(5, 1, 10)` → True, `kozte(10, 1, 10)` → True, `kozte(11, 1, 10)` → False.

4.5. feladat ★☆☆ - Magánhangzó

Írj `maganhangzo(c)` függvényt (a, e, i, o, u). Használd fel arra, hogy megszámold egy beolvasott szó magánhangzóit.

Ellenőrzés: `maganhangzo("a")` → True, `maganhangzo("k")` → False. Az `informatika` szóban 5 magánhangzó van.

4.6. feladat ★☆☆ - Szökőév

Írj `szokoev(ev)` függvényt. A szabály: szökőév, ha osztható 4-gyel; de ha 100-zal is osztható, akkor csak abban az esetben, ha 400-zal is.

Ellenőrzés: `szokoev(2024)` → True, `szokoev(2023)` → False, `szokoev(1900)` → False, `szokoev(2000)` → True.

4.7. feladat ★★★ - Prímszám

Írj `prim(n)` függvényt. **Tipp:** az `osszetett` függvény mintájára dolgozz. Vigyázz: az 1 nem prím.

Ellenőrzés: `prim(2)`, `prim(3)`, `prim(17)` → True; `prim(1)`, `prim(9)`, `prim(15)` → False.

5. Ami a függvényben történik, az ott is marad

A függvényben létrehozott változók csak a függvényen belül léteznek. A paraméter pedig a függvény saját változója.

Kísérlet - ki látja a változót?

```
def szamol():  
    eredmeny = 6 * 7  
  
szamol()  
print(eredmeny)
```

Mi történik? Olvasd el a hibaüzenet utolsó sorát.

Az **eredmeny** a **szamol** függvény változója, ezért a főprogram nem látja. Az ilyet **helyi** (lokális) változónak hívjuk. Amikor a függvény véget ér, a helyi változói megszűnnek.

Tippelj - a paraméter is helyi változó

```
def novel(x):  
    x = x + 1  
    print("A fuggvényben:", x)  
  
a = 5  
novel(a)  
print("A foprogramban:", a)
```

Mit ír ki a két sor? A második meglepő lehet: a függvény csak a saját **x** változóját növelte meg, a főprogram **a** változója nem változott.

A megoldás: a függvény adja vissza az új értéket, a főprogram pedig tegye el.

```
def novel(x):  
    return x + 1  
  
a = 5  
a = novel(a)  
print(a)
```

Tippelj - a listával más a helyzet!

```
def duplaz(lista):  
    for i in range(len(lista)):  
        lista[i] = lista[i] * 2  
  
szamok = [1, 2, 3]  
duplaz(szamok)  
print(szamok)
```

Mit ír ki? Hasonlítsd össze az előző kísérlettel.

Figyelem! Ha a függvény egy lista **elemeit** írja át, az eredeti lista változik meg, mert a függvény ugyanazt a listát kapja meg, nem másolatot. Számmal és sztringgel ez nem fordulhat elő, mert azok nem módosíthatók.

Tippelj - két egyforma nevű változó

```
def szamol():  
    i = 100  
    print(i)  
  
i = 1  
szamol()  
print(i)
```

A két `i` két külön változó, csak a nevük egyezik. Ezért nem kell attól félned, hogy egy függvény „elrontja” a főprogram változóit.

Próbáld ki - két eredmény egyszerre

Pythonban a `return` után vesszővel elválasztva több értéket is visszaadhatsz:

```
def osztas(a, b):  
    return a // b, a % b  
  
hányados, maradek = osztas(17, 5)  
print(hányados, maradek)
```

Feladatok

5.1. feladat ★☆☆ - Javítás

Javítsd ki a fejezet első programját úgy, hogy a `szamol` függvény visszaadja az eredményt, a főprogram pedig kiírja.

Ellenőrzés: a program kimenete `42`.

5.2. feladat ★☆☆ - Mennyi lesz?

Előbb papíron dönts el, mit ír ki ez a program, aztán futtasd.

```
def felez(x):  
    x = x // 2  
  
a = 40  
felez(a)  
felez(a)  
print(a)
```

Ezután javítsd ki `return` használatával úgy, hogy a két felezés tényleg megtörténjen.

Ellenőrzés: a javított program kimenete `10`.

5.3. feladat ★★☆☆ - Csere

Írj `csere(a, b)` függvényt, amely a két értéket fordított sorrendben adja vissza. A főprogramban ezzel cseréld meg két változó értékét.

Ellenőrzés: ha `x = 3` és `y = 8`, akkor az `x, y = csere(x, y)` sor után `x` értéke 8, `y` értéke 3.

5.4. feladat ★★☆☆ - Biztonságos beolvasás

Írj `beker(also, felso)` függvényt. Addig kérjen be számot a felhasználótól, amíg az az `also` és a `felso` közé nem esik (a határok megengedettek). A jó számot adja vissza.

Ellenőrzés: a `jegy = beker(1, 5)` hívásnál írd be sorban: 9, 0, 3. A `jegy` értéke 3 lesz.

6. Függvények listával és sztringgel

Egy függvény paramétere lista vagy sztring is lehet, és ilyet vissza is adhat.

Próbáld ki - lista paraméter

```
def osszeg(lista):
    s = 0
    for x in lista:
        s = s + x
    return s

jegyek = [5, 3, 4, 5, 2]
pontok = [42, 67, 55]

print(osszeg(jegyek))
print(osszeg(pontok))
```

Az összegzést egyszer írtad meg, mégis két különböző listára használod. A listás jegyzet minden mintája (megszámlálás, legnagyobb elem, keresés) megírható így, függvényként.

Tipp: Most már érted, mik azok a `len()`, `sum()`, `max()`: kész függvények, amelyeket valaki megírt helyetted. Ebben a fejezetben a sajátjaidat írod meg, ezért a `sum`, a `max` és a `count` itt csak ellenőrzésre használható.

Próbáld ki - sztringet kap, sztringet ad vissza

```
def fordit(s):
    uj = ""
    for betu in s:
        uj = betu + uj
    return uj

print(fordit("alma"))

szo = "informatika"
vissza = fordit(szo)
print(szo, "-", vissza)
```

Tippelj - új lista vagy a régi?

Ez a két függvény hasonlót csinál, mégis másképp viselkedik. Mit ír ki a program?

```
def duplaz_helyben(lista):
    for i in range(len(lista)):
        lista[i] = lista[i] * 2

def duplazott(lista):
    uj = []
    for x in lista:
        uj.append(x * 2)
    return uj

szamok = [1, 2, 3]
masik = duplazott(szamok)
print(szamok, masik)

duplaz_helyben(szamok)
print(szamok)
```

Az első függvény magát a listát írja át. A második az eredetit békén hagyja, és új listát ad vissza. Mindkettő hasznos; a nevük árulja el, melyik mit csinál.

Feladatok

6.1. feladat ★☆☆ - Kiíró

Írj `kiir(lista)` függvényt, amely a lista elemeit egy sorba írja ki, szóközzel elválasztva, szögletes zárójelek nélkül. A fejezet többi feladatában ezt használd a listák kiírására.

Ellenőrzés: az `[5, 3, 4]` listára a kimenet `5 3 4`.

6.2. feladat ★☆☆ - Ötösök

Írj `hany_otos(lista)` függvényt, amely megszámolja az ötösöket.

Ellenőrzés: az `[5, 3, 4, 5, 2]` listára az eredmény `2`.

6.3. feladat ★☆☆ - Átlag

Írj `atlag(lista)` függvényt. Az összeget ne számold ki újra: hívd meg az `osszeg` függvényt.

Ellenőrzés: az `[5, 3, 4, 5, 2]` listára az eredmény `3.8`.

6.4. feladat ★☆☆ - Legnagyobb

Írj `legnagyobb(lista)` függvényt.

Ellenőrzés: a `[42, 67, 55, 91, 38]` listára `91`, a `[-5, -2, -9]` listára `-2`.

6.5. feladat ★☆☆ - Van-e benne?

Írj `van_benne(lista, keresett)` függvényt, az `in` használata nélkül. **Tipp:** a 4. fejezet `osszetett` függvényének a szerkezetét kövesd.

Ellenőrzés: a `[4, 8, 15]` listában a 8-ra `True`, a 7-re `False` az eredmény.

6.6. feladat ★☆☆ - Betűszámláló

Írj `hanszor(s, c)` függvényt, amely megszámolja, hányszor szerepel a `c` karakter az `s` szövegben.

Ellenőrzés: `hanszor("informatika", "a") → 2`, `hanszor("informatika", "i") → 2`, `hanszor("informatika", "z") → 0`.

6.7. feladat ★☆☆ - Palindrom

Írj `palindrom(s)` függvényt. Használd fel a `fordit` függvényt!

Ellenőrzés: `palindrom("kajak") → True`, `palindrom("alma") → False`.

6.8. feladat ★★ - Nagy kezdőbetűk

Írj `nagy_kezdozetuk(s)` függvényt, amely a szöveg minden szavát nagybetűvel kezdi.

Ellenőrzés: `nagy_kezdozetuk("kiss anna maria") → Kiss Anna Maria`.

6.9. feladat ★★ - Párosak

Írj `parosak(lista)` függvényt, amely egy új listában adja vissza a páros elemeket.

Ellenőrzés: a `[3, 8, 12, 7, 10]` listára az eredmény (a `kiir` függvénnyel kiírva) `8 12 10`.

6.10. feladat ★★ - Mindet növel

Írj `novel_mind(lista, mennyivel)` függvényt, amely magát a listát módosítja: minden elemét megnöveli.

Ellenőrzés: az `[1, 2, 3]` lista a `novel_mind(szamok, 10)` hívás után `11 12 13`.

7. Program függvényekből

Egy nagyobb programot úgy érdemes megírni, hogy minden részfeladat külön függvény. A főprogram ezeket csak sorban meghívja.

Próbáld ki

```
def beolvas(db):
    lista = []
    for i in range(db):
        lista.append(int(input()))
    return lista

def osszeg(lista):
    s = 0
    for x in lista:
        s = s + x
    return s

def atlag(lista):
    return osszeg(lista) / len(lista)

db = int(input("Hany jegy lesz? "))
jegyek = beolvas(db)

print("Összeg:", osszeg(jegyek))
print("Átlag:", atlag(jegyek))
```

Olvasd el csak a főprogramot, vagyis az utolsó négy sort. Érted, mit csinál a program, anélkül hogy a függvények belsejét megnéznéd? Ez a jó program ismertetőjele.

Három jó tanács a saját programjaidhoz:

- Egy függvény egyetlen dolgot csináljon, és a neve mondja meg, mit.
- A számoló függvény adja vissza az eredményt, ne írja ki. A kiírás a főprogram dolga.
- Ne írd meg egyszerre az egész programot. Írj meg egy függvényt, próbáld ki a főprogramból, és csak utána írd a következőt.

Feladatok

7.1. feladat ★★☆☆ - Bővítés

Bővítsd a fenti programot két függvénnyel: a **legnagyobb** a legjobb jegyet adja vissza, a **hany_jobb(lista, hatar)** pedig azt, hány elem nagyobb a határnál. Írd ki a legjobb jegyet és azt, hány jegy jobb az átlagnál.

Ellenőrzés: ha a jegyek 5, 3, 4, 5, 2, akkor az összeg 19, az átlag 3.8, a legjobb jegy 5, és 3 jegy jobb az átlagnál.

7.2. feladat ★★☆☆ - Számológép

Írj négy függvényt: **osszead**, **kivon**, **szoroz**, **oszt**. Mindegyik két számot kap, és az eredményt adja vissza. A főprogram olvasson be egy műveletet ebben a formában: **8 / 2**, és a műveleti jel alapján hívja meg a megfelelő függvényt. **Tipp:** a sort a **split()** három részre bontja; a két számot **float()** alakítja át.

Ellenőrzés: **8 / 2** → 4.0, **3 * 5** → 15.0, **2.5 + 1** → 3.5, **1 - 4** → -3.0.

7.3. feladat ★★☆☆ - Jelszóellenőr függvényekkel

Írj három függvényt: **van_kisbetu(s)**, **van_nagybetu(s)**, **van_szamjegy(s)**. Írj egy negyediket is: az **eros** függvény akkor ad igazat, ha a jelszó legalább 8 karakter, és mindhárom másik függvény igazat ad rá.

Ellenőrzés: **eros("Alma1234")** → True, **eros("alma1234")** → False, **eros("Alma12")** → False.

7.4. feladat ★★★ - Prímek százig

A 4.7. feladat `prim` függvényével írd ki az összes prímszámot 1 és 100 között, és számold meg őket.

Ellenőrzés: a sor így kezdődik: `2 3 5 7 11`, és összesen 25 prímszám van.

7.5. feladat ★★★ - Caesar-rejtjel

Írj `eltol(c, k)` függvényt, amely egy kisbetűt `k` hellyel eltol az ábécében (a végén körbefordulva). Írj `titkosit(s, k)` függvényt is, amely az `eltol` segítségével a szó minden betűjét eltolja.

Ellenőrzés: `titkosit("alma", 3) → dopd`, `titkosit("xyz", 3) → abc`. A visszafejtéshez nem kell új függvény: `titkosit("dopd", 23) → alma`.

7.6. feladat ★★★ - Tökéletes számok

Egy szám tökéletes, ha megegyezik a nála kisebb osztóinak összegével ($6 = 1 + 2 + 3$). Írj `osztok_osszege(n)` és `tokeletes(n)` függvényt, és keresd meg a tökéletes számokat 1 és 1000 között.

Ellenőrzés: `osztok_osszege(12) → 16`. Három tökéletes számot kell találnod: 6, 28, 496.

Hibakereső: mit jelent a hibaüzenet?

A Python hibaüzenetének mindig az **utolsó sorát** olvasd el először. Keresd meg a táblázatban azt a sort, amely hasonlít rá.

Amit látsz	Mit jelent?	Mit tegyél?
<code>NameError: name 'vonal' is not defined</code>	A Python nem ismeri ezt a nevet.	Ha függvény: a hívás a def alatt van? Jól írtad a nevét? Ha változó: egy függvény helyi változója, add vissza return -nel.
<code>TypeError: vonal() missing 1 required positional argument</code>	Kevesebb adatot adtál át, mint ahány paramétere a függvénynek van.	Számold meg a paramétereket a def sorában és a hívásban.
<code>TypeError: vonal() takes 1 positional argument but 2 were given</code>	Több adatot adtál át, mint ahány paraméter van.	Ugyanaz: a két darabszámnak egyeznie kell.
A kiírásban None jelenik meg	Olyan függvény eredményét írod ki, amely nem ad vissza semmit.	Hiányzik a return , vagy print áll a helyén.
<code>TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'</code>	Olyan függvény eredményével számolsz, amely nem ad vissza semmit.	Tegyél return -t a függvénybe.
Más szövegű TypeError	Valószínűleg más típusú adatot adtál át, mint amire a függvény számít (például szöveget szám helyett).	Nézd meg, mit vár a függvény, és mit adtál át a hívásban.
<code>IndentationError: expected an indented block</code>	A def sora után nincs behúzott törzs.	Húzd be a függvény sorait négy szóközzel.
<code>SyntaxError: expected ':'</code>	Lemaradt a kettőspont a def sor végéről.	Pótold a kettőspontot.
UnboundLocalError	A függvényben a főprogram egyik változóját próbálsz módosítani.	Add át paraméterként, az új értéket pedig add vissza return -nel.
Nincs hiba, de nem jelenik meg semmi	A függvényt nem hívtad meg, vagy az eredményét nem írtad ki.	Van hívás a főprogramban? Kitetted a zárójelet? Ott van a print ?
A változó értéke nem változott meg	A függvény csak a saját változóját módosította.	Lásd az 5. fejezetet: return , és az eredmény eltárolása.

Összefoglaló táblázat (puska)

Minta	Jelentés
<code>def nev():</code>	függvény paraméter nélkül
<code>def nev(a, b):</code>	függvény két paraméterrel
<code>return ertek</code>	eredmény visszaadása; a függvény azonnal véget ér
<code>return a, b</code>	két eredmény visszaadása egyszerre
<code>nev(5)</code>	hívás
<code>e = nev(5)</code>	hívás, az eredmény eltárolásával
<code>x, y = nev(5)</code>	két visszaadott érték eltárolása
<code>if nev(5):</code>	logikai függvény használata feltételben

Figyelem! Három dolog, amit soha ne felejts el: a függvényt a hívása előtt kell megírni; a kiírás (`print`) és a visszaadás (`return`) nem ugyanaz; a függvény a számokat nem tudja megváltoztatni a főprogramban, a listák elemeit viszont igen.